

Chapter 1 & Chapter 2

Chapter 1. 머신러닝과 딥러닝

1.1 인공지능, 머신러닝과 딥러닝

- 인공지능(AI)
 - 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터(기계)가 할 수 있도록 하는 기술
 - 구현 방법 : 머신러닝, 딥러닝 → 학습 모델을 제공하여 데이터를 분류할 수 있는 기술
 - 관계 : 인공지능 > 머신러닝 > 딥러닝
- 머신러닝
 - 인간이 데이터 전처리 → 데이터의 특징을 스스로 추출 불가능
 - 각 데이터(혹은 이미지) 특성을 컴퓨터(기계)에 인식시키고 학습시켜 문제 해결
- 딥러닝
 - 인간의 작업 생략 → 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석
- 머신러닝 Vs. 딥러닝

구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘 적용, 예측 수행	정보 전달 신경망 사용, 데이터 특징 및 관계 해석
재사용	다양한 알고리즘 사용, 재사용 불가능	재사용
데이터	수천 개	수백만 개 이상
훈련 시간	단시간	장시간

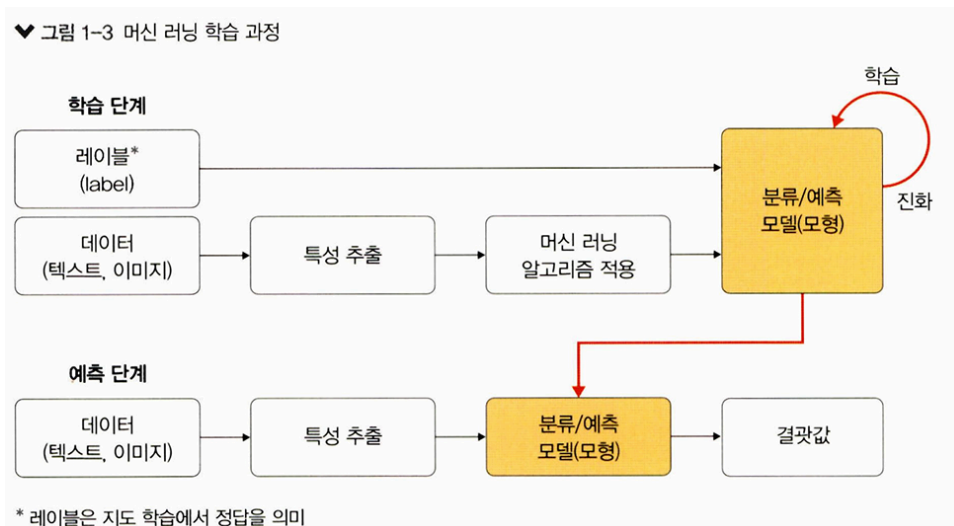
구분	머신 러닝	딥러닝
결과	점수, 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

1.2 머신러닝이란

: 컴퓨터가 학습할 수 있게 하는 알고리즘과 기술 개발 분야

1.2.1 머신 러닝 학습 과정

- 크게 **학습 단계**와 **예측 단계**로 구분



- 학습 단계 : ML 알고리즘에 적용하여 학습 → 이 결과로 모형 생성
- 예측 단계 : 생성된 모형에 새로운 데이터를 적용하여 결과 예측



특성 추출(feature extraction)

- 데이터별 특징 탐색 후 이를 토대로 데이터를 벡터로 변환하는 작업
- ML에서는 데이터의 패턴/규칙을 찾아내려면 사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환 필요

- 머신 러닝 주요 구성 요소

▼ 데이터

- ML이 학습 모델을 만드는 데 사용하는 것
- 데이터의 특징이 잘 반영되고 편향되지 않는 훈련 데이터 확보가 중요
- '훈련 데이터셋', '테스트 데이터셋' 용도로 분리해 사용
(혹은 '훈련 데이터셋'을 또다시 '훈련 데이터셋'과 '검증 데이터셋'으로 분리하여 사용)

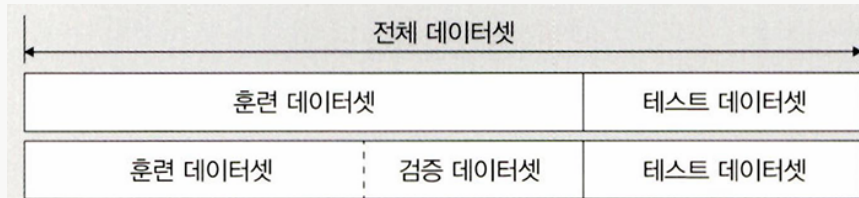
▼ 모델

- ML 학습 단계에서 얻은 최종 결과물의 가설이라고도 함
- 모델의 학습 절차
 1. 모델(또는 가설 선택)
 2. 모델 학습 및 평가
 3. 평가를 바탕으로 모델 업데이트⇒ 이 세 단계를 반복하여 최고의 모델 탐색



훈련과 검증, 테스트 데이터셋

훈련과 테스트 데이터셋으로 분리/ 훈련 데이터셋을 다시 훈련과 검증 용도로 분리 차이?



- 검증 데이터셋을 사용하는 이유: 모델 성능 평가를 위함
 - but 학습 데이터셋의 양이 많지 않다면 검증 데이터 사용하는 건 좋지 않음
- 모델 성능 평가가 필요한 이유
 1. 테스트 데이터셋에 대한 성능 가늠 가능
 2. 모델 성능을 높이는 데 도움(ex. 과적합시)

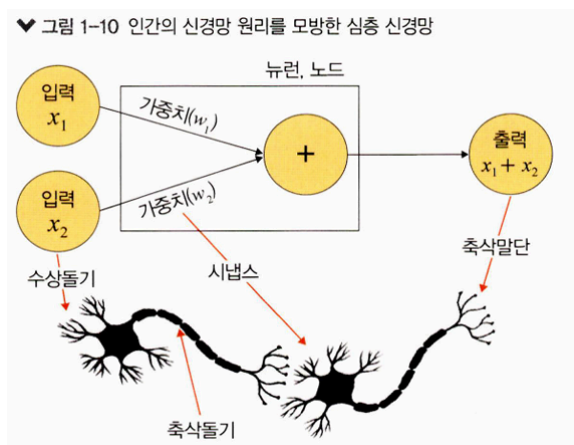
1.2.2 머신 러닝 학습 알고리즘

- 지도학습
 - 정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법
- 비지도 학습
 - 정답 X, 특징이 비슷한 데이터를 클러스터링 하여 예측하는 학습 방법
- 강화 학습
 - 자신의 행동에 대한 보상을 받으며 학습 진행(사례: 쿠키런)
- 자주 사용되는 알고리즘

구분	유형	알고리즘
Supervised learning	Classification	- KNN - SVM - Decision Tree - Logistic Regression
	Regression	Linear Regression
Unsupervised learning	Clustering	- K-means clustering - DBSCAN
	Demensionality Reduction	PCA
Reinforcement learning	-	MDP

1.3 딥러닝이란

: 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 ML 방법의 일종

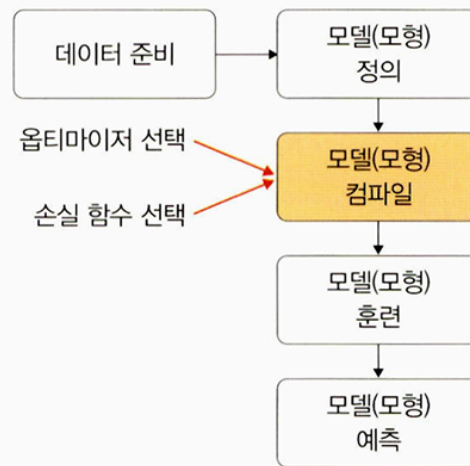


- 인간의 뇌의 뉴런과 시냅스 개념을 컴퓨터에 적용
- 각각의 뉴런은 복잡하게 연결된 수많은 뉴런을 병렬 연산하여 기존 컴퓨터가 수행하지 못했던 음성, 영상 인식 등 처리를 가능하게 함

1.3.1 딥러닝 학습 과정

- 딥러닝 모델의 학습 과정

▼ 그림 1-11 딥러닝 모델의 학습 과정



1. 데이터 준비 : 파이토치, 케라스, 공개 데이터 사용
 2. 모델(모형) 정의 : 신경망 생성
 3. 모델(모형) 컴파일 : 활성화 함수, 손실 함수, 옵티마이저 선택
 4. 모델(모형) 훈련 : 한 번에 처리할 데이터 양 지정
 - 배치와 에포크 선택이 중요, 파라미터와 하이퍼파라미터의 최적 값 찾기
 5. 모델(모형) 예측 : 검증 데이터셋을 생성한 모델에 적용하여 실제 예측 진행
- 딥러닝 학습 과정에서 핵심 구성 요소 : 신경망, 역전파
 - 심층 신경망 : 데이터셋의 어떤 특성이 중요한지 스스로에게 알려주는 기능
 - 역전파 : 가중치 값 업데이트, 미분이 성능에 영향을 미치는 주요 요소

1.3.2 딥러닝 학습 알고리즘

- 지도 학습
 - 이미지 분류: 이미지 또는 비디오상의 객체를 식별하는 컴퓨터 비전 기술
 - 합성곱 신경망(CNN)이 CV에서 많이 사용됨 : 이미지 분류, 인식, 분할
 - 시계열 데이터 분류

- 순환 신경망(RNN) : 역전파 과정에서 기울기 소멸 문제 발생 → LSTM(게이트 3개 추가)으로 문제 개선

• 비지도 학습

- 워드 임베딩 : 단어를 벡터로 표현
 - 워드투벡터, 글로브
 - 자연어 처리 분야의 일종으로 번역이나 음성 인식 등 서비스에서 사용
- 군집 : 아무런 정보가 없는 상태에서 데이터 분류하는 방법
 - 목표: 한 클러스터 내 데이터는 매우 비슷하게 구성, 다른 클러스터와는 구분되게
 - 모델 성능을 높이기 위해 ML 단독 보단 DL과 함께 사용하여 군집 알고리즘 적용

• 전이 학습

- 사전에 학습 완료된 모델로 원하는 학습에 미세 조정 기법을 이용하여 학습
 - 사전 학습 모델: 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습된 모델

• 딥러닝 알고리즘

구분	유형	알고리즘
Supervised learning	이미지 분류	- CNN - AlexNet - ResNet
	시계열 데이터 분석	- RNN - LSTM
Unsupervised learning	군집	- GMM - SOM
	차원 축소	- AutoEncoder - PCA
Transfer learning	전이 학습	- BERT - MobileNetV2
Reinforcement learning	-	MDP

Chapter 2. 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

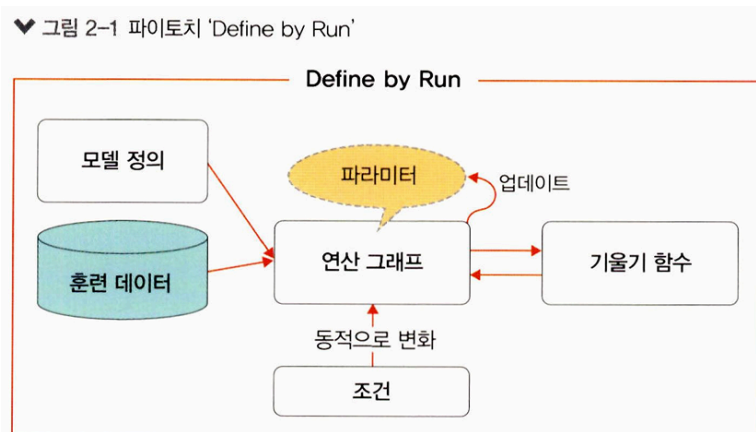
: 딥러닝 프레임워크로 루아 언어로 개발되었던 토치를 페이스북에서 파이썬 버전으로 올림

- 대상
 - 넘파일 대체하면서 GPU를 이용한 연산이 필요한 경우
 - 최대한의 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요한 경우

2.1.1 파이토치 특징 및 장점

GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임 워크

- GPU : 연산 속도를 빠르게 하는 역할
- 텐서
 - 파이토치의 데이터 형태
 - 단일 데이터 형식으로 된 자료들의 다차원 행렬
 - 간단한 명령어(변수 뒤 `.cuda()`를 추가) 사용해서 GPU로 연산 수행 가능
- 동적 신경망 : 훈련 반복할 때마다 네트워크 변경이 가능한 신경망(ex. 학습 중 은닉층 추가)
 - 'Define by Run' 방식 사용: 연산 그래프 정의와 동시에 값 초기화





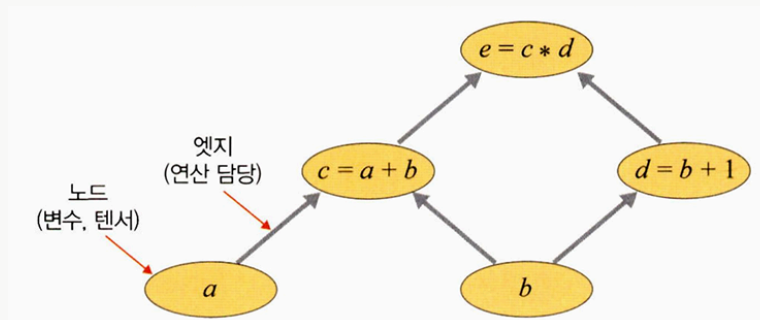
벡터, 행렬, 텐서

- 1차원 축(행) = axis 0 = 벡터
- 2차원 축(열) = axis 1 = 행렬
- 3차원 축(채널) = axis 2 = 텐서 : 행렬의 다차원 표현
 - `torch.tensor()` 사용



연산 그래프

- 방향성O, 노드와 엣지로 구성

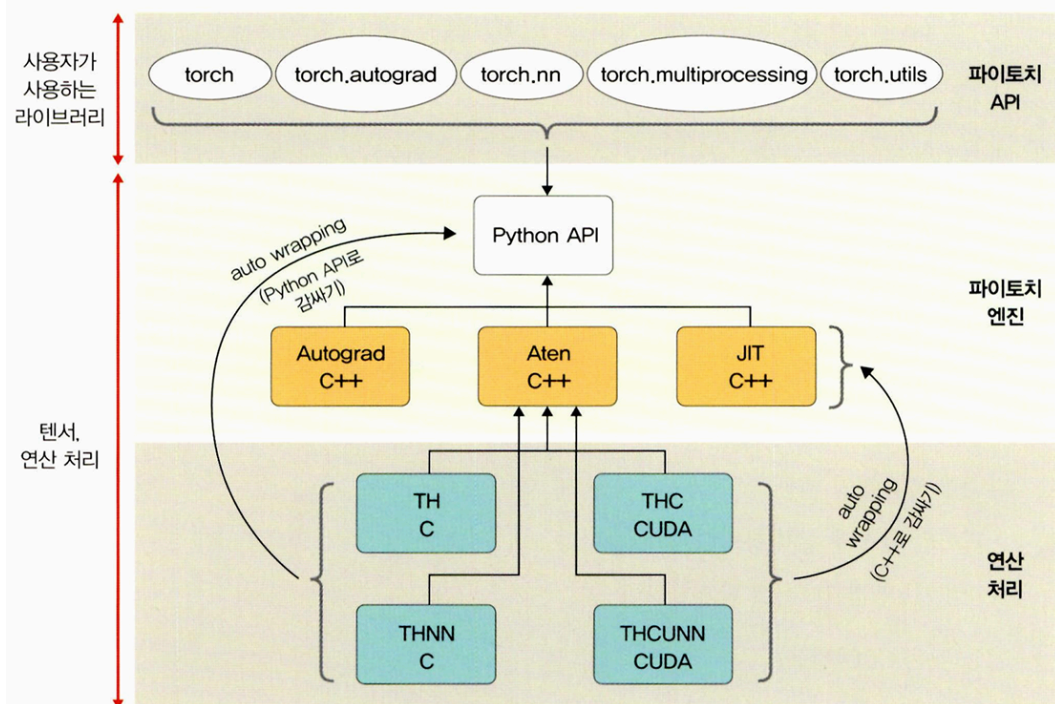


- 신경망은 연산 그래프를 이용하여 계산 수행
 - 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산, 이후 경사 하강법으로 가중치 업데이트

- 파이토치의 장점
 - 단순함(효율적인 계산)
 - 성능(낮은 CPU 활용)
 - 직관적인 인터페이스

2.1.2. 파이토치의 아키텍처

▼ 그림 2-4 파이토치의 아키텍처



• 파이토치 API

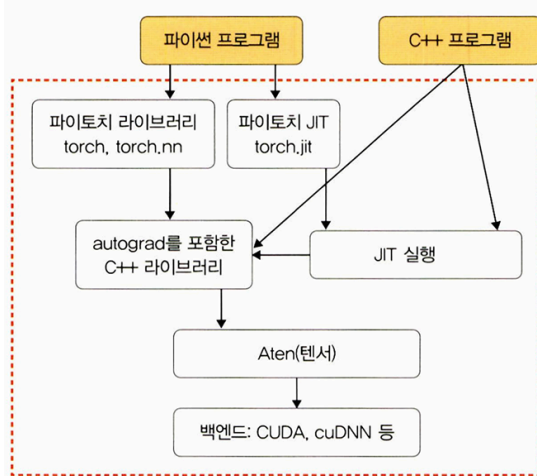
- 사용자가 이해하기 쉬운 API를 제공하여 텐서에 대한 처리, 신경망 구축 및 훈련을 도움
- 사용자 인터페이스 제공 but 실제 계산X (작업 전달 역할만 함)

▼ 사용자 편의성을 위해 제공되는 패키지

- `torch` : GPU를 지원하는 텐서 패키지
- `torch.autograd` : 자동 미분 패키지 - 연산 그래프 즉시 계산
- `torch.nn` : 신경망 구축 및 훈련 패키지
- `torch.multiprocessing` : 파이썬 멀티프로세싱 패키지
- `torch.utils` : DataLoader 및 기타 유틸리티를 제공하는 패키지

• 파이토치 엔진 : 다차원 텐서 및 자동 미분 처리

▼ 그림 2-5 파이토치 엔진



- Autograd C++ : 가중치, 바이어스를 업데이트 하는 과정에서 필요한 미분 자동 계산
- Aten C++ : 텐서 라이브러리 제공
- JIT C++ : 계산 최적화를 위한 JIT(Just In-Time) 컴파일러
- 파이토치 엔진 라이브러리는 C++로 래핑 후 Python API 형태로 제공

• 연산 처리

- C/CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산 수행
- 제공되는 패키지는 CPU와 GPU(TH, THC)를 이용하여 효율적인 데이터 구조, 다차원 텐서에 대한 연산 처리

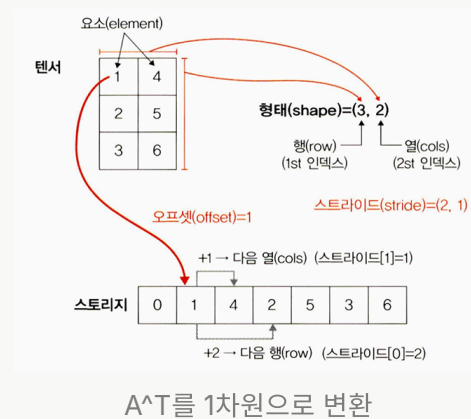
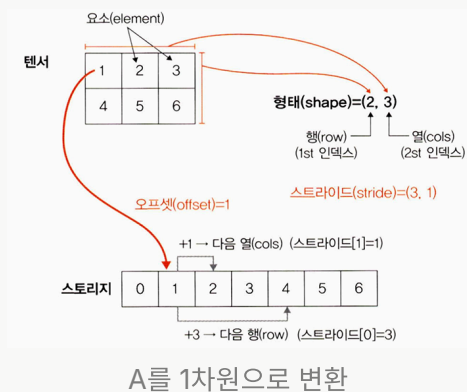


텐서를 메모리에 저장하기

텐서는 N차원이든 메모리 저장 시 1차원 배열 형태 → 1차원 배열 형태여야 메모리 저장

스토리지(storage): 변환된 1차원 배열

- 오프셋(offset): 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스
- 스트라이드(stride): 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수. 요소가 연속적으로 저장되기 때문에 행 중심으로 스트라이드 항상 1
- A 와 A^T 를 구분하는 용도로 오프셋과 스트라이드 사용



⇒ 오프셋과 스트라이드는 데이터 자체가 아닌 행렬/텐서를 구분하기 위해 사용

2.2 파이토치 기초 문법

2.2.1 텐서 다루기

(1) 텐서 생성 및 변환

- 텐서 생성

```
import torch
print(torch.tensor([[1,2],[3,4]])) # 2차원 형태의 텐서 생성
print(torch.tensor([[1,2],[3,4]], device="cuda:0")) # GPU에 텐서 생성
print(torch.tensor([[1,2],[3,4]], dtype=torch.float64)) # dtype을 이용하여
텐서 생성
```

- 텐서 → ndarray 변환

```
temp = torch.tensor([[1,2],[3,4]])
print(temp.numpy()) # 텐서를 ndarray로 변환

temp = torch.tensor([[1,2],[3,4]], device="cuda:0")
print(temp.to("cpu").numpy()) # GPU상의 텐서를 CPU의 텐서로 변환한 후 ndarray로 변환
```

(2) 텐서의 인덱스 조작

- 배열처럼 인덱스를 바로 지정, 슬라이스 등 사용
- 텐서의 자료형
 - torch.FloatTensor : 32비트의 부동 소수점
 - torch.DoubleTensor : 64비트의 부동 소수점
 - torch.LongTensor : 64비트의 부호가 있는 정수

(3) 텐서 연산 및 차원 조작

- 다양한 수학 연산 가능, GPU 사용시 더 빠른 연산
- 텐서 간 타입이 다르면 연산 불가능(ex. FloatTensor과 DoubleTensor 간 사칙 연산)
- 텐서 차원 조작
 - view : 텐서 차원 변경
 - stack : 텐서 결합
 - t, transpose : cat과 차원 교환

2.2.2 데이터 준비

(1) 단순히 파일을 불러와서 사용

- 판다스 라이브러리 이용 → JSON, PDF, CSV 등의 파일 불러오기

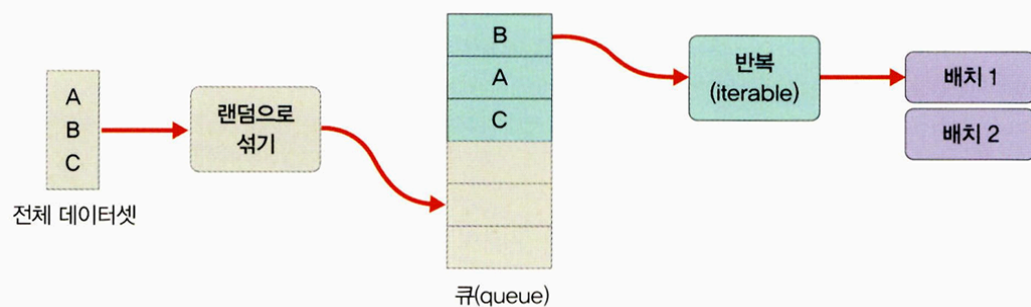
(2) 커스텀 데이터셋을 만들어서 사용

- 데이터를 한번에 다 부르지X, 조금씩 나눠 불러 사용하는 방식
- CustomDataset클래스 구현을 위한 형태

```
class CustomDataset(torch.utils.data.Dataset):  
    def __init__(self): #필요한 변수 선언, 데이터셋 전처리를 해주는 함수  
    def __len__(self): #데이터셋의 길이. 즉, 총 샘플의 수를 가져오는 함수  
    def __getitem__(self, index): #데이터셋에서 특정 데이터를 가져오는 함수(index번째 데이터를 반환하는 함수이며, 이때 반환되는 값은 텐서의 형태를 취해야 함)
```



torch.utils.data.DataLoader



- 학습에 사용될 데이터 전체를 보관, 모델 학습 시 배치 만큼 데이터를 꺼내 사용
- ! 데이터는 내부적으로 반복자에 포함된 인덱스를 이용하여 배치 크기만큼 반환

(3) 파이토치에서 제공하는 데이터셋 사용

- 파이토치에서 제공하는 데이터셋 확인

<https://docs.pytorch.org/vision/0.8/datasets.html>

- 파이토치에서 제공하는 데이터셋을 내려받기 위해 requests라이브러리 설치 필요

2.2.3 모델 정의

파이토치에서 모델을 정의하기 위해 모듈을 상속한 클래스 사용

- 계층(layer): 모듈/모듈을 구성하는 한 개의 계층. 합성곱층, 선형계층 등.
- 모듈(module): 한 개 이상의 계층이 모여서 구성된 것. 모듈이 새 모듈 생성 가능.
- 모델(model): 최종적으로 원하는 네트워크. 한 개의 모듈이 모델이 될 수도.

(1) 단순 신경망을 정의하는 방법

- nn.Module을 상속받지X. 단순한 모델. 구현이 쉬움

(2) nn.Module()을 상속하여 정의하는 방법

- 파이토치에서 nn.Module()을 상속받은 모델을 `__init__()`과 `Forward()` 함수 포함
 - `__init__()` : 모듈(nn.Linear, nn.Conv2d), 활성화 함수 등 정의
 - `forward()` : 모델에서 실행 돼야 하는 연산 정의

(3) Sequential 신경망을 정의하는 방법

- `__init__()`에서 사용할 네트워크 모델 정의
- `forward()` 함수에서는 모델에서 실행되는 계산을 좀 더 가독성이 있게 코드 작성
- Sequential 객체는 그 안에 포함된 각 모듈을 순차적으로 실행
- nn.Sequential은 모델의 계층이 복잡할수록 효과가 뛰어남



model.modules() & model.children()

model.modules()는 모델의 네트워크에 대한 모든 노드 반환,
model.children()은 같은 수준(level)의 하위노드 반환

(4) 함수로 신경망을 정의하는 방법

- (3)과 같이 Sequential 이용, but 함수 선언 시 변수에 저장해 놓은 계층들 재사용 가능
- 모델이 복잡해지는 단점 → 복잡한 모델: 함수 < nn.Module() 상속받아 사용이 더 편리
- ReLU, Softmax 및 Sigmoid와 같은 활성화 함수는 모델 정의 시 지정

2.2.4 모델의 파라미터 정의

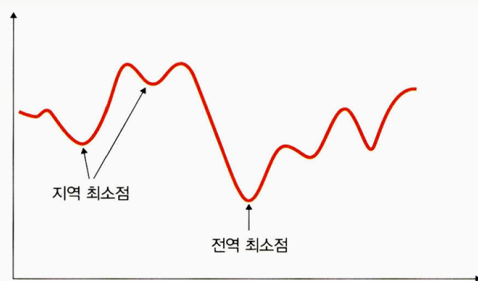
사전에 정의할 파라미터

파라미터	설명	특징
손실함수(loss function)	학습하는 동안 출력과 실제 값 사이의 오차 측정	- BCELoss: 이진 분류를 위해 사용 - CrossEntropyLoss: 다중 클래스 분류를 위해 사용 - MSELoss : 회귀 모델에서 사용
옵티마이저(optimizer)	데이터와 손실 함수를 바탕으로 모델의 업데이트 방법 결정	- step()을 통해 전달받은 파라미터 업데이트 - 모델의 파라미터별로 다른 기준 적용 - torch.optim.Optimizer(params, defaults) : 모든 옵티마이저의 기본이 되는 클래스 - zero_grad(): 옵티마이저에서 사용된 파라미터의 기울기를 0으로 만들 - torch.optim.lr_scheduler: 에포크에 따라 학습률 조절
학습률 스케줄러(learning rate scheduler)	- 미리 지정한 횟수의 에포크를 지날 때마다 학습률 감소 - 학습 초기에 빠른 학습 진행, 전역 최소점에 다다르면 학습률을 줄여 최적점으로	- optim.lr_scheduler.LambdaLR : 람다 함수 이용, 함수 결과를 학습률 설정 - optim.lr_scheduler.StepLR : 특정 단계마다 학습률을 감마 비율만큼 감소 - optim.lr_scheduler.MultistepLR: 지정된 에포크에만 감마 비율로 감소 - optim.lr_scheduler.ExponentialLR: 에포크마다 이전 학습률에 감마만큼 곱함

파라미터	설명	특징
		- optim.lr_scheduler.CosineAnnealingLR: 학습률을 코사인 함수 형태처럼 변화 - optim.lr_scheduler.ReduceLROnPlateau: 학습이 잘 되고 있는지 아닌지에 따라 동적으로 학습률을 변화
지표(metrics)	훈련과 테스트 단계 모니터링	



전역 최소점과 최적점



- 전역 최소점: 오차가 가장 작을 때의 값(최적점)
- 지역 최소점: 전역 최소점을 찾아가는 과정에서 만나는 홀. 옵티마이저가 지역 최소점에서 학습을 멈추면 최솟값을 갖는 오차 찾을 수 X

2.2.5 모델 훈련

학습 시킨다 : $y=wx+b$ 라는 함수에서 적절한 w 와 b 값 찾기(전역 최소점에 이를 때까지)

- 훈련 방법

▼ 그림 2-12 파이토치 학습 절차

딥러닝 학습 절차	파이토치 학습 절차
모델, 손실 함수, 옵티마이저 정의	모델, 손실 함수, 옵티마이저 정의
전방향 학습(입력 → 출력 계산)	optimizer.zero_grad(): 전방향 학습, 기울기 초기화
손실 함수로 출력과 정답의 차이(오차) 계산	output = model(input): 출력 계산
역전파 학습(기울기 계산)	loss = loss_fn(output, target): 오차 계산
기울기 업데이트	loss.backward(): 역전파 학습
	optimizer.step(): 기울기 업데이트
	모델 학습 과정

1. optimizer.zero_grad()
이용: 기울기 초기화

cf) loss.backward() :
기울기 값 계산, 누적 계산 방식 → 누적 계산 필요 X 모델에는 불필요
2. loss.backward() : 기울기 자동 계산

2.2.6 모델 평가

- 모델에 대한 평가 방법
 1. 함수 이용
 2. 모듈 이용
 3. 이외에 사이킷런에서 제공한 혼동 행렬 이용하는 방법 등

2.2.7 훈련 과정 모니터링

텐서보드를 이용하기!

⇒ 파라미터 값의 변화를 시각화하여 살펴보고, 성능 추적 혹은 평가 용도로 사용 가능!

- 텐서 보드 사용방법
 1. 텐서보드 설정
 2. 텐서보드에 기록
 3. 텐서보드를 사용하여 모델 구조 살펴보기



model.train() & model.eval()

- model.train() : 훈련 데이터셋 사용, 모델 훈련. 드롭아웃 활성화
- model.eval() : 검증, 테스트 데이터셋 사용(모든 노드 사용한다는 의미)
 - with torch.no_grad()를 사용하는 이유?
검증/테스트 과정에서 역전파 필요X → 기울기 값 저장하지 않도록

2.3 실습 환경 설정

-생략-

2.4 파이토치 코드 맛보기

Week1_예습과제_허정인.ipynb